

고객을 세그먼테이션하자 [프로젝트] - 이정숙

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
SELECT *  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`  
  
Limit 10;
```

Job information Results Visualization JSON Execution details Execution graph									
Row	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom	
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom	
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom	
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom	
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom	
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom	
10	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom	

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT COUNT(*) AS total_rows  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`  
  
Limit 10;
```

Query results		
Job information	Results	Visualization
JSON	Execution details	Execution graph
Row	total_rows	
1	541909	

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
SELECT
COUNT(InvoiceNo) AS InvoiceNo_countnt,
COUNT(StockCode) AS StockCode_count,
COUNT(Description) AS Description_count,
COUNT(Quantity) AS Quantity_countnt,
COUNT(InvoiceDate) AS InvoiceDate_count,
COUNT(UnitPrice) AS UnitPrice_count,
COUNT(CustomerID) AS CustomerID_countnt,
COUNT(Country) AS Country_count,

FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
;
```

Job information		Results	Visualization	JSON	Execution details	Execution graph		
Row	InvoiceNo_countnt	StockCode_count	Description_count	Quantity_countnt	InvoiceDate_count	UnitPrice_count	CustomerID_coun...	Country_count
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```

SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`

UNION ALL

SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 E
ND) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
;

```

Row	column_name ▼	missing_percenta...
1	Description	0.27
2	CustomerID	24.93

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```

SELECT DISTINCT Description
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`

WHERE StockCode = '85123A';

```

Row	Description ▼	
1	WHITE HANGING HEART T-LIGHT HOLDER	
2	?	
3	wrongly marked carton 22804	
4	CREAM HANGING HEART T-LIGHT HOLDER	

결측치 처리

- **DELETE** 구문을 사용하며, **WHERE** 절을 통해 데이터를 제거할 조건을 제시

```
DELETE FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]];
```

Job information	Results	Execution details	Execution graph
i This statement removed 135,080 rows from data.			

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, **COUNT**가 1보다 큰 데이터를 세어보기

```
SELECT COUNT(*) AS duplicate_rows
FROM (
  SELECT
    InvoiceNo,
    StockCode,
```

```

        Description,
        Quantity,
        InvoiceDate,
        UnitPrice,
        CustomerID,
        Country,
        COUNT(*) AS cnt
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`
GROUP BY
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country
HAVING COUNT(*) > 1
);

```

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	duplicate_rows ▾				
1	4837				

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기
 - CREATE OR REPLACE TABLE** 구문을 활용하여 모든 컬럼(*)을 **DISTINCT** 한 데이터로 업데이트

```
CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
;
```

Job information

Results

Execution details

Execution graph



This statement replaced the table named data.

```
72
73 SELECT COUNT(*) AS duplicate_rows
74 FROM (
75   SELECT
76     InvoiceNo,
77     StockCode,
78     Description,
79     Quantity,
80     InvoiceDate,
81     UnitPrice,
82     CustomerID,
83     Country,
84     COUNT(*) AS cnt
85   FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
86   GROUP BY
87     InvoiceNo,
88     StockCode,
89     Description,
90     Quantity,
91     InvoiceDate,
92     UnitPrice,
93     CustomerID,
94     Country
95   HAVING COUNT(*) > 1
96 );
97
98
```

✓ Query completed

Using on-demand processing quota

Query results

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	duplicate_rows
1	0

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo 의 개수를 출력하기

```
SELECT COUNT(DISTINCT InvoiceNo) AS Count_InvoiceNo
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
;
```

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	Count_InvoiceNo
1	22190

- 고유한 InvoiceNo 를 앞에서부터 100개를 출력하기

```
SELECT DISTINCT InvoiceNo
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
LIMIT 100;
```

Row	InvoiceNo	
1	541431	
2	C541433	
3	537626	
4	542237	
5	549222	
6	556201	
7	562032	
8	573511	
9	581180	
10	539318	
11	541998	
12	548955	
13	568172	
14	577609	
15	543037	
16	544156	
17	545323	
18	545332	

Results per page: 50 1 – 50 of 100 < >

- **InvoiceNo** 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT DISTINCT *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

Row	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
6	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
7	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
8	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
9	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
10	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
11	C547388	22645	CERAMIC HEART FAIRY CAKE M...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT ROUND(SUM(CASE WHEN Quantity < 0 THEN 1 ELSE 0 END)/COUNT(*) * 100, 1) AS Canceled_Ratio
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
```

```
-- 선호 버전:
-- SELECT ROUND(COUNTIF(Quantity <0)/COUNT(*) * 100,1) AS Canceled_Ratio
-- FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
```

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Canceled_Ratio				
1	2.2				

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기

```
SELECT COUNT(DISTINCT StockCode)
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`;
```

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	f0_				
1	3684				

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

Job information		Results	Visualization	JS
Row	StockCode	sell_cnt		
1	85123A	2065		
2	22423	1894		
3	85099B	1659		
4	47566	1409		
5	84879	1405		
6	20725	1346		
7	22720	1224		
8	POST	1196		
9	22197	1110		
10	23203	1108		

- StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
```

```

r'[0-9]', '')) AS number_count
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
)
WHERE number_count = 0 or number_count = 1;

```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	StockCode	number_count				
1	POST	0				
2	M	0				
3	C2	1				
4	D	0				
5	BANK CHARGES	0				
6	PADS	0				
7	DOT	0				
8	CRUK	0				

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```

SELECT
  ROUND(
    SUM(CASE
      WHEN LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) <= 1
      THEN 1 ELSE 0
    END) / COUNT(*) * 100,
    2
  ) AS percentage

```

```
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
`;
```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	percentage					
1	0.48					

- 제품과 관련되지 않은 거래 기록을 제거하기

```
DELETE FROM project_name.modulabs_project.data
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    # [[YOUR QUERY]]
  );
```

-- DELETE 대신 CREATE OR REPLACE TABLE 로 실행. 같은 결과 값.

```
CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.data` AS
SELECT *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
WHERE LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
r'[0-9]', '')) > 1;
```

```
171
172 SELECT *
173 FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
174 WHERE LENGTH(StockCode) <= LENGTH(REGEXP_REPLACE(StockCode, '[0-9]', '')) <= 1;
175
```

✓ Query completed

Using on-demand processing quota

Query results

Job information **Results** Visualization JSON Execution details Execution graph

i There is no data to display.

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT DISTINCT Description, COUNT(*) AS description_cnt
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

Row	Description	description_cnt
1	WHITE HANGING HEART T-LIGHT HOLDER	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORNAMENT	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY DESIGN	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE CASES	1062
10	SPOTTY BUNTING	1026
11	PAPER CHAIN KIT 50'S CHRISTMAS	1013
12	LUNCH BAG SPACEBOY DESIGN	1006
13	LUNCH BAG CARS BLUE	1000
14	HEART OF WICKER SMALL	990
15	NATURAL SLATE HEART CHALKBOARD	989
16	JAM MAKING SET WITH JARS	966
17	LUNCH BAG PINK POLKADOT	961
18	LUNCH BAG SUKI DESIGN	932
19	ALARM CLOCK BAKELIKE RED	917
20	REX CASH+CARRY JUMBO SHOPPER	900
21	WOODEN PICTURE FRAME WHITE FINISH	900
22	JUMBO BAG PINK POLKADOT	887

Results per page: 50 1 – 30 of 30

• 서비스 관련 정보를 포함하는 행들을 제거하기

```
SELECT DISTINCT Description
FROM project_name.modulabs_project.data
WHERE REGEXP_CONTAINS(Description, r'[a-z]');
```

```
-- High Resolution Image → 상품 사진 제공 서비스
-- Next Day Carriage → 익일 배송 서비스
```

```
DELETE
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
WHERE Description LIKE '%Carriage%'
      OR Description LIKE '%Image%';

-- 혹은
-- WHERE REGEXP_CONTAINS(Description, r'Carriage|Image');
```

< Job information Results Visualization JSON Execution details >		
Row	Description ▾	
1	BAG 250g SWIRLY MARBLES	
2	3 TRADITIONAL BISCUIT CUTTE...	
3	BAG 125g SWIRLY MARBLES	
4	POLYESTER FILLER PAD 30CMx...	
5	BAG 500g SWIRLY MARBLES	
6	POLYESTER FILLER PAD 45x45...	
7	POLYESTER FILLER PAD 40x40...	
8	ESSENTIAL BALM 3.5g TIN IN E...	
9	FRENCH BLUE METAL DOOR SI...	
10	POLYESTER FILLER PAD 45x30...	
11	NUMBER TILE VINTAGE FONT ...	
12	NUMBER TILE COTTAGE GARD...	
13	POLYESTER FILLER PAD 65CMx...	
14	FLOWERS HANDBAG blue and ...	
15	FOLK ART GREETING CARD,pac...	
16	THE KING GIFT BAG 25x24x12cm	
17	POLYESTER FILLER PAD 60x40...	
Results per page: 50 ▾ 1 – 17 of 17 < < > >		

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
```

Job information		Results	Visualization	JSON	Execution details	Execution graph		
Row	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingd...	MEDIUM CERAMIC TOP STORAGE JAR
2	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingd...	MEDIUM CERAMIC TOP STORAGE JAR
3	537626	22771	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	CLEAR DRAWER KNOB ACRYLIC EDWARDIAN
4	537626	84997D	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	PINK 3 PIECE POLKADOT CUTLERY SET
5	537626	22375	4	2010-12-07 14:57:00 UTC	4.25	12347	Iceland	AIRLINE BAG VINTAGE JET SET BROWN
6	537626	22773	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	GREEN DRAWER KNOB ACRYLIC EDWARDIAN
7	537626	21064	6	2010-12-07 14:57:00 UTC	5.95	12347	Iceland	BOOM BOX SPEAKER BOYS
8	537626	22494	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	EMERGENCY FIRST AID TIN
9	537626	22726	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE GREEN
10	537626	22725	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE CHOCOLATE
11	537626	22492	36	2010-12-07 14:57:00 UTC	0.65	12347	Iceland	MINI PAINT SET VINTAGE
12	537626	22805	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	BLUE DRAWER KNOB ACRYLIC EDWARDIAN
13	537626	21731	12	2010-12-07 14:57:00 UTC	1.65	12347	Iceland	RED TOADSTOOL LED NIGHT LIGHT
14	537626	22728	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE PINK

UnitPrice 살펴보기

- UnitPrice 의 최솟값, 최댓값, 평균을 구하기

```
SELECT
  ROUND(MIN(UnitPrice),2) AS min_price,
  ROUND(MAX(UnitPrice),2) AS max_price,
  ROUND(AVG(UnitPrice),2) AS avg_price
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
```

Job information Results Visualization JSON Execution details Execution graph					
Row	// min_price ▼	// max_price ▼	// avg_price ▼	//	
1	0.0	649.5	2.9		

- 단가가 0원인 거래의 개수, 구매 수량(**Quantity**)의 최솟값, 최댓값, 평균 구하기

```
SELECT
SUM(Quantity) AS cnt_quantity,
MIN(Quantity) AS min_quantity,
MAX(Quantity) AS max_quantity,
AVG(Quantity) AS avg_quantity

FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
WHERE UnitPrice = 0;
```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	cnt_quantity	min_quantity	max_quantity	avg_quantity		
1	13877	1	12540	420.5151515151...		

- **UnitPrice = 0** 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.data` AS
SELECT *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
WHERE UnitPrice > 0
```

```

236 SELECT *
237 FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
238 WHERE UnitPrice = 0;

```

✓ Query completed

Using on-demand processing quota

Query results [+ Cr](#)

Job information **Results** Visualization JSON Execution details Execution graph

i There is no data to display.

11-7. RFM 스코어

Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```

SELECT
DATE(InvoiceDate) AS InvoiceDay, *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`

```

Job information		Results	Visualization	JSON	Execution details	Execution graph			
Row	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
3	2010-12-07	537626	21731	12	2010-12-07 14:57:00 UTC	1.65	12347	Iceland	RED TOADSTOOL LED NIGHT LI...
4	2010-12-07	537626	84997C	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	BLUE 3 PIECE POLKADOT CUTL...
5	2010-12-07	537626	22494	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	EMERGENCY FIRST AID TIN

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```

SELECT
  MAX(InvoiceDate) over() AS most_recent_date,
  DATE(InvoiceDate) AS InvoiceDay,
  *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`

```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate
1	2011-12-09 12:50:00 UTC	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC
2	2011-12-09 12:50:00 UTC	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC
3	2011-12-09 12:50:00 UTC	2010-12-07	537626	21731	12	2010-12-07 14:57:00 UTC
4	2011-12-09 12:50:00 UTC	2010-12-07	537626	84997C	6	2010-12-07 14:57:00 UTC
5	2011-12-09 12:50:00 UTC	2010-12-07	537626	22494	12	2010-12-07 14:57:00 UTC

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```

SELECT
  CustomerID,
  MAX(InvoiceDate) AS most_recent_date
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
GROUP BY CustomerID

```

Job information		Results	Visualization	JSC
Row	CustomerID	most_recent_date		
1	12346	2011-01-18 10:17:00 UTC		
2	12347	2011-12-07 15:52:00 UTC		
3	12348	2011-09-25 13:13:00 UTC		
4	12349	2011-11-21 09:51:00 UTC		
5	12350	2011-02-02 16:01:00 UTC		
6	12352	2011-11-03 14:37:00 UTC		
7	12353	2011-05-19 17:47:00 UTC		
8	12354	2011-04-21 13:11:00 UTC		
9	12355	2011-05-09 13:49:00 UTC		
10	12356	2011-11-17 08:40:00 UTC		
11	12357	2011-11-06 16:07:00 UTC		
12	12358	2011-12-08 10:26:00 UTC		
13	12359	2011-12-02 11:21:00 UTC		
14	12360	2011-10-18 15:22:00 UTC		

- 가장 최근 일자(**most_recent_date**)와 유저별 마지막 구매일(**InvoiceDay**)간의 차이를 계산하기

```

SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
);

```

Row	CustomerID	recency
1	12782	4
2	13082	126
3	13127	77
4	13227	201
5	13298	1
6	13555	73
7	13558	5
8	13755	3
9	13846	24
10	13870	2
11	13881	3
12	13979	73

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```
CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.user_r` AS

SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
--DATE_DIFF(MAX(InvoiceDay) OVER(), InvoiceDay, DAY) AS recency

FROM (
  SELECT
    CustomerID,
    MAX(InvoiceDate) AS InvoiceDay
  FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
```

```
GROUP BY CustomerID  
);
```

Job information		Results	Visualization	
Row	CustomerID	recency		
1	17315	0		
2	12662	0		
3	17001	0		
4	18102	0		
5	15910	0		
6	12680	0		
7	17581	0		
8	16446	0		
9	15311	0		
10	12433	0		

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```
SELECT  
  CustomerID,  
  COUNT(InvoiceNo) AS purchase_cnt  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da  
ta`  
GROUP BY CustomerID;
```

Row	CustomerID	purchase_cnt
1	12346	2
2	12347	182
3	12348	27
4	12349	72
5	12350	16
6	12352	84
7	12353	4
8	12354	58
9	12355	13
10	12356	58
11	12357	131
12	12358	17
13	12359	251
14	12360	126

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
-- 아이템별로 :
-- SELECT
--   CustomerID,
--   StockCode, SUM(Quantity) AS item_cnt
--   FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
--   GROUP BY CustomerID, StockCode;

SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
  FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
  GROUP BY CustomerID;
```

Job information		Results	Visualization	JSC
Row	CustomerID	StockCode	item_cnt	
1	12347	22375	28	
2	12347	22492	108	
3	12347	84558A	126	
4	12347	22423	15	
5	12347	20719	40	
6	12347	22371	32	
7	12347	23076	240	
8	12347	23084	84	
9	12347	23506	40	
10	12347	23508	40	
11	12348	21982	144	
12	12348	84991	144	
13	12348	21977	144	
14	12348	21213	144	
15	12348	22952	144	

Row	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573
11	12357	2708

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```

-- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐
서 user_rf라는 이름의 테이블에 저장하기
CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.user_rf` AS
-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT InvoiceNo) AS purchase_cnt
    FROM `project-3e5836df-194c-42fe-b66.modulabs_project.
data`
    GROUP BY CustomerID
),
-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
    SELECT
        CustomerID,
        SUM(Quantity) AS item_cnt
    FROM `project-3e5836df-194c-42fe-b66.modulabs_project.
data`
    GROUP BY CustomerID
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
    pc.CustomerID,
    pc.purchase_cnt,
    ic.item_cnt,
    ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
    ON pc.CustomerID = ic.CustomerID
JOIN `project-3e5836df-194c-42fe-b66.modulabs_project.user

```

```

_r` AS ur
ON pc.CustomerID = ur.CustomerID;

```

Row	CustomerID	purchase_cnt	item_cnt	recency
12	16569	1	93	3
13	17914	1	457	3
14	12442	1	181	3
15	16528	1	171	3
16	12478	1	233	3
17	15318	1	642	3
18	17383	1	148	4
19	16597	1	184	4
20	13790	1	748	4
21	14219	1	78	4
22	15097	1	170	4
23	12367	1	172	4
24	18015	1	157	4
25	16535	1	356	5
26	17936	1	257	5
27	15773	1	311	5
28	13153	1	418	5
29	16988	1	43	5
30	13560	1	63	7
31	17942	1	38	7
32	13017	1	48	7
33	18174	1	50	7
34	12587	1	50	7

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

거래금액 = Quantity × UnitPrice

↓

고객별로 합계

↓

소수점 첫째 자리 반올림

총 지출액 = ROUND(SUM(Quantity × UnitPrice), 1)

GROUP BY CustomerID

SELECT

CustomerID,

ROUND(SUM(Quantity × UnitPrice), 1) AS user_total

FROM `project-3e5836df-194c-42fe-b66.modulabs\_project.data`

GROUP BY CustomerID

Row	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.6
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4

- 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기

-- 기준 테이블 user_rf: 고객별 purchase_cnt, item_cnt, recency

-- 붙일 테이블: data, 총거래금액인 'user_total' SUM(Quantity *

```

UnitPrice)
-- 조인 키: CustomerID
-- 조인 방식: LEFT JOIN
-- 고객별 평균 거래 금액 계산: 'user_average' = user_total/ purchase_cnt
-- 최종 테이블 이름: user_rfm

CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.user_rfm` AS
SELECT
    rf.CustomerID AS CustomerID,
    rf.purchase_cnt,
    rf.item_cnt,
    rf.recency,
    ut.user_total,
    ROUND(ut.user_total/ rf.purchase_cnt, 1) AS user_average
-- 고객별 평균 거래 금액 계산

FROM `project-3e5836df-194c-42fe-b66.modulabs_project.user_rfm` AS rf
LEFT JOIN (
    -- 고객 별 총 지출액
    SELECT
        CustomerID,
        ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
    FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
    GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;

-- "고객별 총 지출액을 계산해 보세요. 소수점 첫째 자리에서 반올림하세요."

```

RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

```
SELECT*
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.user
_rfm`
```

Row	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.6	794.6
2	15520	1	314	1	343.5	343.5
3	13298	1	96	1	360.0	360.0
4	14569	1	79	1	227.4	227.4
5	13436	1	76	1	196.9	196.9
6	15195	1	1404	2	3861.0	3861.0
7	14204	1	72	2	150.6	150.6
8	15471	1	256	2	454.5	454.5
9	14578	1	240	3	168.6	168.6
10	16569	1	93	3	124.2	124.2
11	12650	1	250	3	242.4	242.4
12	15318	1	642	3	312.6	312.6

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) **user_rfm** 테이블과 결과를 합치기
- 3) **user_data** 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH unique_products AS (
```

```

SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
FROM project_name.modulabs_project.data
GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_rfm AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;

```

Row	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products
1	14911	242	76823	1	128768.2	532.1	1791
2	12748	217	23516	0	29820.0	137.4	1767
3	17841	169	22613	1	39861.5	235.9	1330
4	14606	125	5932	1	11486.6	91.9	829
5	15311	118	37673	0	59284.2	502.4	571
6	13089	118	30742	2	57322.1	485.8	636
7	12971	88	9204	3	10933.8	124.2	94
8	13408	75	16128	1	27888.4	371.8	234
9	14646	73	196556	1	278778.0	3818.9	699
10	16029	66	33687	38	60369.9	914.7	43
11	14156	64	56896	9	113685.8	1776.3	714
12	13798	63	23515	1	36352.9	577.0	115
13	16422	60	33584	17	34311.0	571.9	70
14	18102	60	64124	0	259657.3	4327.6	150
15	14527	57	2082	2	8477.5	148.7	220

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 군 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합

```

CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH purchase_intervals AS (
    -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수

```

```

SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_interval
FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
        CustomerID,
        DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DAY) AS interval_
    FROM
        project_name.modulabs_project.data
    WHERE CustomerID IS NOT NULL
)
GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;

```

Row	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval
1	13068	2	200	10	344.0	172.0	1	309.0
2	18084	2	311	16	87.5	43.8	2	284.0
3	18080	2	642	18	1231.5	615.8	2	223.0
4	12875	2	2019	143	343.2	171.6	1	219.0
5	14777	2	-9	4	-17.4	-8.7	2	176.5
6	18273	3	80	2	204.0	68.0	1	127.5
7	17029	2	400	110	716.0	358.0	1	126.0
8	14865	2	108	7	52.2	26.1	4	121.33
9	14987	2	128	15	326.4	163.2	2	116.33
10	18017	3	300	81	523.0	174.3	1	114.5
11	16532	3	2016	17	6748.8	2249.6	2	106.67
12	17616	3	224	173	571.2	190.4	1	96.5
13	14520	2	146	1	270.6	135.3	3	96.33
14	16308	4	8000	37	2560.0	640.0	1	74.0
15	14609	4	7026	72	601.6	150.4	2	74.0
16	16462	4	12	64	102.0	25.5	1	72.67
17	18246	2	166	23	596.1	298.1	3	69.67
18	13848	3	3700	92	1255.0	418.3	3	68.25
19	12908	3	600	176	360.0	120.0	1	68.0
20	16446	3	2	0	2.9	1.0	3	68.0
21	17026	2	97	185	154.8	77.4	4	61.33
22	17929	5	107	11	834.5	166.9	4	59.2
23	14295	2	83	40	157.7	78.8	6	58.4
24	14548	2	256	150	652.8	326.4	2	55.67

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기

1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수

2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율

- 취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data`에 통합하기
(취소 비율은 소수점 두번째 자리)

-- 2. 평균 구매 주기

-- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)

-- 균 구매 소요 일수를 계산하고, 그 결과를 `user\_data`에 통합

```

CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_interval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DAY) AS interval_
    FROM
      `project-3e5836df-194c-42fe-b66.modulabs_project.data`
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;

```

Row	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	13017	1	48	7	204.0	204.0	1	0.0	1	0	0.0
2	13747	1	8	373	79.6	79.6	1	0.0	1	0	0.0
3	16078	1	16	283	79.2	79.2	1	0.0	1	0	0.0
4	15753	1	144	304	79.2	79.2	1	0.0	1	0	0.0
5	17763	1	12	263	15.0	15.0	1	0.0	1	0	0.0
6	12353	1	20	204	89.0	89.0	4	0.0	1	0	0.0
7	14373	1	17	358	76.5	76.5	4	0.0	1	0	0.0
8	16190	1	88	40	215.1	215.1	6	0.0	1	0	0.0
9	13723	1	43	217	199.9	199.9	6	0.0	1	0	0.0
10	15256	1	55	148	98.5	98.5	6	0.0	1	0	0.0
11	13866	1	57	74	145.7	145.7	7	0.0	1	0	0.0
12	15717	1	70	234	150.3	150.3	7	0.0	1	0	0.0
13	15775	1	68	245	104.9	104.9	7	0.0	1	0	0.0
14	13806	1	56	63	256.4	256.4	10	0.0	1	0	0.0
15	12837	1	24	173	134.1	134.1	12	0.0	1	0	0.0
16	15678	1	184	52	352.7	352.7	12	0.0	1	0	0.0
17	13441	1	204	91	296.6	296.6	13	0.0	1	0	0.0
18	14638	1	166	50	443.6	443.6	16	0.0	1	0	0.0
19	14476	1	110	257	193.0	193.0	17	0.0	1	0	0.0
20	13803	1	121	255	277.1	277.1	17	0.0	1	0	0.0
21	14413	1	67	362	121.0	121.0	22	0.0	1	0	0.0

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 **user_data** 를 출력하기

```
SELECT*
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.user_data`
```

[결과 이미지를 넣어주세요]

Row	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	13017	1	48	7	204.0	204.0	1	0.0	1	0	0.0
2	13747	1	8	373	79.6	79.6	1	0.0	1	0	0.0
3	16078	1	16	283	79.2	79.2	1	0.0	1	0	0.0
4	15753	1	144	304	79.2	79.2	1	0.0	1	0	0.0
5	17763	1	12	263	15.0	15.0	1	0.0	1	0	0.0
6	12353	1	20	204	89.0	89.0	4	0.0	1	0	0.0
7	14373	1	17	358	76.5	76.5	4	0.0	1	0	0.0
8	16190	1	88	40	215.1	215.1	6	0.0	1	0	0.0
9	13723	1	43	217	199.9	199.9	6	0.0	1	0	0.0
10	15256	1	55	148	98.5	98.5	6	0.0	1	0	0.0
11	13866	1	57	74	145.7	145.7	7	0.0	1	0	0.0
12	15717	1	70	234	150.3	150.3	7	0.0	1	0	0.0
13	15775	1	68	245	104.9	104.9	7	0.0	1	0	0.0
14	13806	1	56	63	256.4	256.4	10	0.0	1	0	0.0
15	12837	1	24	173	134.1	134.1	12	0.0	1	0	0.0
16	15678	1	184	52	352.7	352.7	12	0.0	1	0	0.0
17	13441	1	204	91	296.6	296.6	13	0.0	1	0	0.0
18	14638	1	166	50	443.6	443.6	16	0.0	1	0	0.0
19	14476	1	110	257	193.0	193.0	17	0.0	1	0	0.0
20	13803	1	121	255	277.1	277.1	17	0.0	1	0	0.0
21	14413	1	67	362	121.0	121.0	22	0.0	1	0	0.0

회고

[회고 내용을 작성해주세요]

Keep : 전체 프로세스를 훑어봄

Problem : RFM 부분 숙지가 필요함

Try : 연습문제로 단계 반복